Trabalho Integrador

Sistema de Biblioteca com JWT e Refresh Token

Projetos base:

jwt-api (back-end) · jwt-app (front-end)

O que vamos construir

Evoluir **jwt-api** e **jwt-app** até virarem um sistema de gestão de biblioteca:

- Login com **access token + refresh token**
- Cadastro de **categorias, autores, livros e leitores**
- Gestão de **atendentes** (perfis admin x atendente)
- Controle de **empréstimos e devoluções** com multa

O que vocês vão aprender

- Organizar a API em módulos: `router → controller → service → model`
- Relacionamentos no Sequelize (1:N e N:N)
- Autenticação stateless com JWT, **renovação** e **revogação**
- Proteção de rotas por perfil (401 x 403)
- Front-end que renova o token **sem deslogar** o usuário

Ponto de partida: o que já existe

jwt-api

- Express 5 + Sequelize + SQLite
- `POST /v1/auth/login` → token de 1h
- `POST /v1/auth/logout` → blacklist
- CRUD básico de `User` (`isAdmin`)

O que falta

- CORS (o front não conecta!)
- Refresh token
- Entidades da biblioteca
- Regras de negócio
- Controle por perfil

Passo 1 · Clonar

```
mkdir biblioteca && cd biblioteca
git clone https://github.com/ADS-Unipar/jwt-api.git
git clone https://github.com/ADS-Unipar/jwt-app.git
```

```
biblioteca/
├── jwt-api/   ← back-end (porta 3000)
└── jwt-app/   ← front-end
```

> 💡 Faça **fork** de cada repositório e clone o fork para poder dar `git push`.

Passo 2 · Subir a API

```
cd jwt-api
npm install
cp .envex .env        # Windows: copy .envex .env
```

```
JWT_SECRET=troque-por-uma-chave-grande-e-aleatoria
JWT_REFRESH_SECRET=outra-chave-diferente-da-primeira
PORT=3000
```

```
npm start   # SERVIDOR RODANDO http://localhost:3000
```

Passo 3 · Liberar o CORS

Sem CORS, o navegador **bloqueia** as chamadas do front (outra porta).

```
npm install cors
```

```
// index.js — antes das rotas
const cors = require('cors');
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
```

Ajuste `origin` para a porta do seu front (Vite 5173, CRA 3001, Expo 8081…).

Passo 4 · Corrigir o bug do login

Hoje a senha é comparada **antes** de checar se o usuário existe → e-mail inexistente derruba a API.

```
// auth/auth.service.js
const user = await userService.findByEmail(email);
if (!user) throw new Error('Invalid email or password');

const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) throw new Error('Invalid email or password');
```

Passo 5 · Testar a API

```
# cria o primeiro admin
curl -X POST http://localhost:3000/v1/user \
  -H "Content-Type: application/json" \
  -d '{"name":"Admin","username":"admin","email":"admin@biblioteca.com",
       "password":"123456","isAdmin":true}'

# login
curl -X POST http://localhost:3000/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@biblioteca.com","password":"123456"}'
```

Resposta `{ "token": "eyJ..." }` → API funcionando ✅

Passo 6 · Conectar o front-end

```
cd ../jwt-app
npm install
```

Aponte a URL da API para `http://localhost:3000/v1` (`.env` , `src/services/api.js` ...):

```
VITE_API_URL=http://localhost:3000/v1
```

```
npm run dev
```

Faça login e confira no **DevTools** → **Network** que `/v1/auth/login` voltou **200**.

Problemas comuns

Sintoma	Solução
Erro de CORS no console	Revise o passo 3 (`origin`)
`secretOrPrivateKey must have a value`	Crie o `.env` com `JWT_SECRET`
Login com e-mail errado derruba a API	Corrija o bug do passo 4
`ECONNREFUSED` no front	API parada ou porta errada
Dados "fantasmas" no banco	Apague `database.sqlite` e coloque no `.gitignore`

Perfis de usuário

👑 Administrador

`isAdmin = true`

- Gerencia atendentes
- Categorias, autores e livros
- Relatórios / dashboard

👨‍💼 Atendente

`isAdmin = false`

- Cadastra leitores
- Registra empréstimos e devoluções
- Consulta o acervo

Entidades

Entidade	Campos mínimos	Relação
User	name, username, email, password, isAdmin, active	1:N Loan
Category	name (único), description	1:N Book
Author	name, nationality, birthDate	N:N Book
Book	title, isbn, year, totalCopies, availableCopies	N:1 Category · N:N Author
Reader	name, cpf (único), email, phone	1:N Loan
Loan	loanDate, dueDate, returnDate, status, fine	N:1 Book, Reader, User
RefreshToken	token, userId, expiresAt, revoked	N:1 User

Regras de negócio

1. Só empresta com `availableCopies > 0` (empréstimo −1, devolução +1)
2. Prazo padrão: **7 dias**
3. Máx. **3 empréstimos ativos** por leitor; com atraso, não empresta
4. Multa de **R$ 1,00 por dia** de atraso
5. Status: `ATIVO`, `DEVOLVIDO`, `ATRASADO`
6. Não exclui registro com vínculo ativo → **409 Conflict**
7. Empréstimo e devolução em **transação** (`db.transaction`)
8. Atendente com `active = false` não faz login

Por que refresh token?

Hoje

- 1 token de **1 hora**
- Expirou → usuário deslogado
- Token roubado vale 1h inteira

Com refresh

- **Access**: curto (15 min)
- **Refresh**: longo (7 dias), salvo no banco
- Renova sozinho, sem novo login
- Dá para **revogar** a sessão

Fluxo de autenticação

```
1. Login             → accessToken (15 min) + refreshToken (7 dias)
2. Requisições       → Authorization: Bearer <accessToken>
3. Access expira     → API responde 401
4. Front intercepta  → POST /auth/refresh { refreshToken }
5. API rotaciona     → revoga o refresh antigo, devolve um novo par
6. Front repete      → refaz a requisição original
7. Refresh inválido  → limpa a sessão e vai para o login
```

Requisitos da API · autenticação

- `POST /auth/login` → `{ accessToken, refreshToken, user }`
- `POST /auth/refresh` → valida no banco (existe, não revogado, não expirado) e **rotaciona**
- Refresh revogado reutilizado → revoga **todos** do usuário (roubo de token)
- `POST /auth/logout` → revoga o refresh token
- `GET /auth/me` → usuário logado
- Middleware `isAdmin` → **403**
- Token inválido/expirado → **401** (hoje responde 400!)

Requisitos do front · autenticação

- Guardar os tokens e explicar os **riscos** da escolha no README
- **Interceptor** (Axios ou `fetch`): recebeu 401 → refresh → repete
- Várias requisições com 401 ao mesmo tempo → **um único** refresh, as demais em **fila**
- Telas protegidas redirecionam para o login
- Esconder menus de admin para atendentes
  (a API também precisa bloquear!)

Endpoints mínimos

Método	Rota	
POST	`/v1/auth/login` · `/refresh` · `/logout`	Autenticação
GET	`/v1/auth/me`	Usuário logado
CRUD	`/v1/user`	Atendentes **(A)**
CRUD	`/v1/categories` · `/v1/authors` · `/v1/books`	Acervo (escrita **A**)
GET	`/v1/books?title=&categoryId=&authorId=&available=`	Busca
CRUD	`/v1/readers` · `/v1/readers/:id/loans`	Leitores
POST / PATCH	`/v1/loans` · `/v1/loans/:id/return`	Empréstimo / devolução
GET	`/v1/reports/dashboard`	Totais **(A)**

(A) = somente administrador · `POST /v1/user` deixa de ser público → crie um **seed** do admin

Organização da API

```
jwt-api/
├── auth/         router, controller, service, middleware, refreshToken.model
├── user/
├── category/
├── author/
├── book/
├── reader/
├── loan/
├── config/db.js
├── models/index.js   ← associações (hasMany, belongsTo, belongsToMany)
└── index.js
```

Mesmo padrão que já existe em `auth/` e `user/`.

Telas do front-end

1. **Login**
2. **Dashboard** (livros, ativos, atrasados, multas)
3. **Categorias** e **Autores**
4. **Livros** com busca e filtros
5. **Leitores** + histórico
6. **Empréstimos** + devolução com multa
7. **Atendentes** (só admin)
8. **Logout** + usuário no topo

Etapas de entrega

Etapa	Conteúdo
1	Projetos conectados, CORS e bug corrigidos, modelagem com associações
2	CRUDs de categoria, autor, livro e leitor testados no Insomnia/Postman
3	Access + refresh com rotação, middleware admin, empréstimo/devolução
4	Front-end completo com interceptor + apresentação

O que entregar

- Links dos **forks** com commits de **todos** os integrantes
- `README.md` em cada projeto (instalação, variáveis, admin do seed, decisões)
- Coleção do Insomnia/Postman exportada (`.json`)
- `.env` **fora** do repositório, `.envex` atualizado
- **Vídeo de até 5 min**: login, token renovando sozinho, empréstimo, devolução com multa, atendente bloqueado

> 💡 Para o vídeo, deixe o access token com **30 segundos** de validade.

Critérios de avaliação

Critério	Peso
Autenticação: login, refresh com rotação, logout, 401/403	**25%**
Regras de empréstimo/devolução (estoque, limite, multa, transação)	**20%**
Modelagem e associações Sequelize	**15%**
CRUDs completos e validações	**15%**
Front-end: integração, interceptor, telas	**15%**
Organização, README, commits e apresentação	**10%**

Desafios extras (até +1 ponto)

- Refresh token em **cookie httpOnly**
- **Reserva** de livro sem exemplar, com fila
- **Paginação** (`?page=&limit=`)
- Validação com **Joi** ou **Zod**
- Documentação com **Swagger**
- Testes com **Jest + Supertest**

Bom trabalho! 📚

Dúvidas? Tragam para a aula.

jwt-api · jwt-app

FORMS para entregar:
https://docs.google.com/forms/d/e/1FAIpQLSeTAjykCBQUpVYCxHnPUsDLr7LvNNgJFuFRVVC3uJ-TGLqPkQ/viewform?usp=sharing&ouid=118074898012931674492